

1. Regression Concepts

Linear Regression Overview:

- Maps input $x \in \mathbb{R}^D$ to target $y \in \mathbb{R}$ using weights w
- Basic form: $y = w_0 + w_1x_1 + \dots + w_Dx_D$
- Uses least squares loss to find optimal weights
- "Bias trick": Add $x_0 = 1$ to make notation cleaner: $y = w^T x$

Key Considerations:

- Basis functions ($\phi(x)$): Transform inputs to capture non-linear relationships
- Regularization: Prevent overfitting by penalizing large weights
- Bias-variance tradeoff: Balance between underfitting and overfitting

Regularization Types:

- Ridge (L2): $\frac{\lambda}{2} w^T w$ - Shrinks all weights toward zero
- Lasso (L1): $\lambda \|w\|_1$ - Creates sparse solutions (some weights = 0)
- Elastic Net: Combination of L1 and L2

Bayesian View:

- Model noise: $y_n \sim \mathcal{N}(w^T x_n, \beta^{-1})$
- Prior on weights: $w \sim \mathcal{N}(0, S_0^{-1})$
- MAP estimation equivalent to ridge regression with $\lambda = \frac{\sigma}{\beta}$

1.1 Ridge Regression Derivation

When to use: Use ridge regression when you suspect multicollinearity in your features or need to prevent overfitting in high-dimensional data.

Objective: Find w_{ridge} that minimizes OLS with L2 regularization.

1. Start with ridge loss function:

$$L(w) = \frac{1}{2} \sum_{n=1}^N (y_n - w^T \phi_n)^2 + \frac{\lambda}{2} w^T w \quad (1)$$

2. In matrix form with design matrix Φ and target y :

$$L(w) = \frac{1}{2} (y - \Phi w)^T (y - \Phi w) + \frac{\lambda}{2} w^T w \quad (2)$$

3. Take gradient, set to zero:

$$\nabla L(w) = -\Phi^T y + \Phi^T \Phi w + \lambda w = 0 \quad (3)$$

$$\Rightarrow (\Phi^T \Phi + \lambda I) w = \Phi^T y \quad (4)$$

4. Optimal weights: $w_{ridge} = (\Phi^T \Phi + \lambda I)^{-1} \Phi^T y$

Comparison with OLS: OLS solution is $w^* = (\Phi^T \Phi)^{-1} \Phi^T y$. Ridge adds λI to $\Phi^T \Phi$, which:

- Makes matrix inversion more stable
- Shrinks weights toward zero
- Reduces overfitting by introducing bias

1.2 Maximum Likelihood Estimation

When to use: Use MLE when you want to find parameters that make your observed data most probable under your chosen model.

General MLE Procedure:

1. Write likelihood function (probability of data given parameters): $p(D|\theta) = \prod_{n=1}^N p(x_n|\theta)$
2. Take logarithm for easier optimization: $\log p(D|\theta) = \sum_{n=1}^N \log p(x_n|\theta)$
3. Find parameters maximizing log-likelihood: $\frac{\partial}{\partial \theta} \log p(D|\theta) = 0$

Example: Linear Regression MLE

1. Assume $y_n \sim \mathcal{N}(w^T x_n, \beta^{-1})$
2. Log-likelihood: $\log p(y|X, w, \beta) = \frac{N}{2} \log \beta - \frac{N}{2} \log(2\pi) - \frac{\beta}{2} \sum_n (y_n - w^T x_n)^2$
3. Take derivative, set to zero: $\frac{\partial \log p}{\partial w} = \beta \sum_n (y_n - w^T x_n) x_n = 0$
4. Solve with matrix notation: $X^T y = X^T X w \Rightarrow w^* = (X^T X)^{-1} X^T y$

Key insight: MLE for linear regression with Gaussian noise is equivalent to minimizing the least squares loss!

1.3 Maximum A Posteriori Estimation

When to use: Use MAP when you have prior knowledge about parameters and want to incorporate this information into your estimation.

MAP General Procedure:

1. Write posterior using Bayes' rule: $p(\theta|D) \propto p(D|\theta)p(\theta)$
2. Take logarithm: $\log p(\theta|D) = \log p(D|\theta) + \log p(\theta) + \text{const.}$
3. Maximize log posterior: $\frac{\partial}{\partial \theta} [\log p(D|\theta) + \log p(\theta)] = 0$

Example: Bayesian Linear Regression

1. Gaussian prior: $w \sim \mathcal{N}(0, S_0^{-1})$
 2. Log posterior: $\log p(w|X, y, \beta) \propto -\frac{\beta}{2} \sum_n (y_n - w^T x_n)^2 - \frac{1}{2} w^T S_0 w$
 3. Take derivative, set to zero: $\beta \sum_n (y_n - w^T x_n) x_n - S_0 w = 0$
 4. Matrix form solution: $w_{MAP} = (\beta X^T X + S_0)^{-1} \beta X^T y$
- Note:** With $S_0 = \lambda I$, we get ridge regression: $w_{MAP} = (X^T X + \frac{\lambda}{\beta} I)^{-1} X^T y$

1.4 Posterior Parameters & Predictive Distribution

When to use: Use posterior predictive distribution when you want to make predictions that account for uncertainty in your parameters.

Finding Posterior Parameters (Conjugate Distributions):

1. Identify prior distribution and parameters
2. Identify likelihood function
3. Use conjugacy to determine posterior form
4. Derive posterior parameters by comparing coefficients

Example: Gaussian-Gaussian conjugacy

- Prior: $w \sim \mathcal{N}(\mu_0, \Sigma_0)$
- Likelihood: $y|x, w \sim \mathcal{N}(w^T x, \sigma^2)$
- Posterior: $w|D \sim \mathcal{N}(\mu_N, \Sigma_N)$ where:

$$\Sigma_N^{-1} = \Sigma_0^{-1} + \frac{1}{\sigma^2} \sum_n x_n x_n^T \quad (5)$$

$$\mu_N = \Sigma_N (\Sigma_0^{-1} \mu_0 + \frac{1}{\sigma^2} \sum_n y_n x_n) \quad (6)$$

Posterior Predictive Distribution:

$$p(y^*|x^*, D) = \int p(y^*|x^*, \theta) p(\theta|D) d\theta \quad (7)$$

For Gaussian posterior: $y^*|x^*, D \sim \mathcal{N}(\mu_N^T x^*, \sigma^2 + x^{*T} \Sigma_N x^*)$

Key insight: The posterior predictive variance includes both noise variance σ^2 and parameter uncertainty $x^{*T} \Sigma_N x^*$.

2. Classification Concepts

Classification Overview:

- Maps input x to discrete class labels y
- Can be binary (two classes) or multi-class
- Decision boundaries separate different classes

Common Classification Methods:

- Discriminant functions: Direct mapping from inputs to class decisions
- Probabilistic discriminative models: Model $p(y|x)$ directly (logistic regression)
- Probabilistic generative models: Model $p(x, y)$ together (Naive Bayes)

Loss Functions:

- 0/1 Loss: Simple but non-differentiable, counts misclassifications
- Logistic Loss: Smooth approximation of 0/1 loss, used in logistic regression
- Hinge Loss: Used in SVMs, penalizes points close to or on wrong side of margin

Training Techniques:

- Gradient Descent: Iterative optimization to minimize loss
- Batch GD: Updates using all training examples
- SGD: Updates using one example at a time, faster but noisier

Generative vs. Discriminative:

- Discriminative: Models decision boundary directly
- Generative: Models how data is generated, can generate new samples

- Naive Bayes: Assumes feature independence given class

2.1 Logistic Regression Gradients

When to use: Use logistic regression when you need probabilistic class membership for binary or multi-class classification with a linear decision boundary.

Binary Logistic Regression:

1. Model: $p(y=1|x) = \sigma(w^T x) = \frac{1}{1+e^{-w^T x}}$
2. Likelihood: $p(\{y_i\}_{i=1}^N | w) = \prod_{i=1}^N \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$ where $\hat{y}_i = \sigma(w^T x_i)$
3. Negative log-likelihood (cross-entropy): $E(w) = -\sum_{i=1}^N \{y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)\}$
4. Gradient: $\nabla E(w) = \sum_{i=1}^N (\hat{y}_i - y_i) x_i$

Multi-class with Softmax: $\nabla_{w_j} E(W) = \sum_{i=1}^N (\hat{y}_{ij} - y_{ij}) x_i$ where $\hat{y}_{ij} = \text{softmax}_j(W x_i)$

Softmax function: $\text{softmax}_k(z) = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}$

2.2 Generative Models for Classification

When to use: Use generative models when you want to model the data distribution, generate new samples, or when you have prior knowledge about how the data is generated.

Framework:

1. Define class prior $p(C_k)$ and class-conditional $p(x|C_k)$
2. Joint distribution: $p(x, C_k) = p(C_k)p(x|C_k)$
3. Full log-likelihood for binary classification: $\log p(\pi, \mu_1, \mu_2, \Sigma) = \sum_{i=1}^N y_i \log[\pi \mathcal{N}(x_i | \mu_1, \Sigma)] + (1 - y_i) \log[(1 - \pi) \mathcal{N}(x_i | \mu_2, \Sigma)]$
4. Maximize to find optimal parameters

MLE Solutions (Gaussian with shared covariance):

$$\pi = \frac{N_1}{N} \quad (8)$$

$$\mu_k = \frac{1}{N_k} \sum_{i: y_i=k} x_i \quad (9)$$

$$\Sigma = \frac{1}{N} \sum_{i=1}^N [y_i(x_i - \mu_1)(x_i - \mu_1)^T + (1 - y_i)(x_i - \mu_2)(x_i - \mu_2)^T] \quad (10)$$

$$+ (1 - y_i)(x_i - \mu_2)(x_i - \mu_2)^T] \quad (11)$$

Using Lagrangian Method: 1. Identify constraints (e.g., $\sum_k \pi_k = 1$) 2. Form Lagrangian: $\mathcal{L}(\theta, \lambda) = \log p(D|\theta) + \lambda(c - \sum_k \pi_k)$ 3. Take derivatives w.r.t. parameters and Lagrange multipliers 4. Solve the resulting system

2.3 Naive Bayes

When to use: Use Naive Bayes when features can reasonably be assumed to be conditionally independent given the class, or when you have limited training data.

Key Idea: Assume features are conditionally independent given the class:

$$p(x|y = C_k) = \prod_{d=1}^D p(x_d|y = C_k) \quad (12)$$

Classification Rule:

$$\hat{y} = \arg \max_k p(y = C_k|x) = \arg \max_k p(y = C_k) \prod_{d=1}^D p(x_d|y = C_k) \quad (13)$$

Parameter Estimation:

- Class priors: $p(y = C_k) = \frac{N_k}{N}$
- For discrete features: $p(x_d = v|y = C_k) = \frac{\text{count}(x_d=v, y=C_k)}{\text{count}(y=C_k)}$
- For continuous features: Often assume Gaussian $p(x_d|y = C_k) = \mathcal{N}(\mu_{dk}, \sigma_{dk}^2)$

Advantages:

- Fast training and prediction
- Works well with small datasets
- Handles missing values naturally

Limitations:

- Independence assumption often violated
- Cannot learn feature interactions

2.4 Gradient Descent Implementation

When to use: Use gradient descent when direct analytical solutions aren't possible or practical, especially with non-convex or high-dimensional loss functions.

Basic Gradient Descent:

1. Initialize parameters $\theta^{(0)}$
2. Choose learning rate η
3. For iterations $t = 0, 1, 2, \dots$:
 - Compute gradient: $\nabla L(\theta^{(t)})$
 - Update: $\theta^{(t+1)} = \theta^{(t)} - \eta \nabla L(\theta^{(t)})$
 - Check convergence

For Logistic Regression: $w^{(t+1)} = w^{(t)} - \eta \sum_{i=1}^N (\sigma(w^{(t)T} x_i) - y_i) x_i$

Stochastic Gradient Descent (SGD): Update using one or a batch of samples: $w^{(t+1)} = w^{(t)} - \eta \sum_{i \in \text{batch}} (\sigma(w^{(t)T} x_i) - y_i) x_i$

Learning Rate Selection:

- Too large: May diverge or oscillate
- Too small: Slow convergence
- Adaptive rates: Decrease over time or use algorithms like Adam, RMSprop

Convergence Criteria:

- Small parameter change: $\|\theta^{(t+1)} - \theta^{(t)}\| < \epsilon$
- Small gradient: $\|\nabla L(\theta^{(t)})\| < \epsilon$
- Maximum iterations reached

4. Support Vector Machines Concepts

SVM Overview:

- Maximum margin classifier
- Finds hyperplane that maximally separates classes
- Uses support vectors (points closest to decision boundary)
- Can handle linearly and non-linearly separable data

Key Components:

- Decision function: $f(x) = w^T x + w_0$
- Margin: Distance from decision boundary to nearest data points
- Support vectors: Data points on margin boundaries
- Regularization parameter C: Controls trade-off between margin size and misclassification

Types of SVMs:

- Hard-margin: No misclassifications allowed, requires linearly separable data

able data

- Soft-margin: Allows some misclassifications, more robust to outliers
- Kernel SVMs: Transform data to higher dimensions for non-linear boundaries

Common Kernels:

- Linear: $K(x, z) = x^T z$
- Polynomial: $K(x, z) = (1 + x^T z)^q$
- RBF (Gaussian): $K(x, z) = \exp(-\gamma \|x - z\|^2)$

Comparison to Logistic Regression:

- SVM: Maximizes margin, focuses on boundary points
- Logistic regression: Probabilistic outputs, influenced by all points

4.1 Maximum Margin Classification

When to use: Use hard-margin SVM when your data is linearly separable and you want a decision boundary with maximum distance to all classes.

Hard Margin Formulation:

$$\min_{w, w_0} \frac{1}{2} \|w\|^2 \quad (14)$$

$$\text{subject to } y_i(w^T x_i + w_0) \geq 1, \forall i \quad (15)$$

Interpretation:

- Decision boundary: $w^T x + w_0 = 0$
- Margin width = $\frac{2}{\|w\|}$
- Support vectors exactly satisfy $y_i(w^T x_i + w_0) = 1$

Soft Margin Formulation:

$$\min_{w, w_0, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \quad (16)$$

$$\text{subject to } y_i(w^T x_i + w_0) \geq 1 - \xi_i, \xi_i \geq 0, \forall i \quad (17)$$

Slack Variables ξ_i :

- $\xi_i = 0$: Point correctly classified beyond margin
- $0 < \xi_i \leq 1$: Point correctly classified but inside margin
- $\xi_i > 1$: Point misclassified

Parameter C:

- Large C: Less regularization, small margin, fewer misclassifications
- Small C: More regularization, large margin, more misclassifications

4.2 The Kernel Trick

When to use: Use kernel SVMs when your data is not linearly separable in the original feature space but may be separable in a higher-dimensional space.

Key Insight: The SVM dual formulation only requires inner products between data points, which can be replaced with kernel functions.

Dual Formulation:

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x_i^T x_j \quad (18)$$

$$\text{subject to } \sum_{i=1}^N \alpha_i y_i = 0, \alpha_i \geq 0, \forall i \quad (19)$$

Decision Function:

$$f(x) = \sum_{i=1}^N \alpha_i y_i K(x_i, x) + w_0 \quad (20)$$

Common Kernels:

- Linear: $K(x, z) = x^T z$
- Polynomial: $K(x, z) = (1 + x^T z)^q$
 - For $q = 2$, includes all constant, linear, and quadratic terms
- RBF: $K(x, z) = \exp(-\gamma \|x - z\|^2)$
 - Corresponds to infinite-dimensional feature space
 - Parameter γ controls width of Gaussian

Mercer's Condition: A valid kernel function must produce a positive semi-definite Gram matrix.

Kernel Construction Rules:

- Sum: $K_1 + K_2$
- Product: $K_1 \cdot K_2$
- Scalar: $c \cdot K$
- Exponentiation: $\exp(K)$

6. Mixture Models Concepts

Mixture Models Overview:

- Probabilistic models for data from multiple sources/distributions
- Each data point belongs to a latent (unobserved) class
- Models joint distribution $p(x, z)$ of data x and latent class z
- Can be used for clustering, density estimation, and generative modeling

General Form:

$$p(x) = \sum_{k=1}^K \pi_k p(x|z = k) \quad (21)$$

where π_k are mixing coefficients ($\sum_k \pi_k = 1$) and $p(x|z = k)$ are component distributions.

Estimation Challenge:

- Maximum likelihood: $\log p(X) = \sum_{i=1}^N \log \sum_{k=1}^K \pi_k p(x_i|z = k)$
- Sum inside logarithm prevents direct optimization
- Solution: Use Expectation-Maximization (EM) algorithm

Types of Mixture Models:

- Gaussian Mixture Models (GMMs): Component distributions are Gaussian
- Mixture of Multinomials: For discrete data (text, genomics)
- Mixture of Experts: Different predictive models for different regions
- Latent Dirichlet Allocation: Documents as mixtures of topics

6.1 Expectation-Maximization (EM) Algorithm

When to use: Use EM when you have latent variable models where direct maximum likelihood is intractable, especially for mixture models.

General Procedure:

1. Initialize parameters $\theta^{(0)}$
2. Repeat until convergence:
 - E-step: Compute $Q(\theta|\theta^{(t)}) = \mathbb{E}_{Z|X, \theta^{(t)}} [\log p(X, Z|\theta)]$
 - M-step: $\theta^{(t+1)} = \arg \max_{\theta} Q(\theta|\theta^{(t)})$

Theoretical Foundation:

- EM maximizes a lower bound (ELBO) on the log likelihood:

$$\log p(X|\theta) \geq \mathbb{E}_{Z|X, \theta^{(t)}} [\log p(X, Z|\theta)] - \mathbb{E}_{Z|X, \theta^{(t)}} [\log p(Z|X, \theta^{(t)})] \quad (22)$$

- E-step: Compute posterior distribution of latent variables
- M-step: Maximize expected complete-data log likelihood
- Guaranteed to increase log likelihood at each iteration

Mathematical Properties:

- Converges to local maximum of likelihood
- ELBO becomes tight when using true posterior
- Different initializations may lead to different local maxima

Applications Beyond Mixture Models:

- Hidden Markov Models
- Latent factor models (PCA, factor analysis)
- Missing data imputation

6.2 Gaussian Mixture Models (GMM)

When to use: Use GMMs when your data appears to come from multiple Gaussian distributions or when you need soft clustering with uncertainty estimates.

Model: $p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$

EM Algorithm for GMM:

1. E-step: Calculate responsibilities (posterior probabilities)

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i|\mu_j, \Sigma_j)} \quad (23)$$

2. M-step: Update parameters

$$N_k = \sum_{i=1}^N \gamma_{ik} \quad (24)$$

$$\pi_k = \frac{N_k}{N} \quad (25)$$

$$\mu_k = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} x_i \quad (26)$$

$$\Sigma_k = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^T \quad (27)$$

GMM vs. K-means:

- GMM provides soft assignments (probabilities)
- GMM models covariance structure
- GMM is more flexible but requires more parameters
- K-means is a special case of GMM with:
 - Equal mixing coefficients $\pi_k = 1/K$
 - Spherical covariances $\Sigma_k = \epsilon I$
 - $\epsilon \rightarrow 0$ (hard assignments)

Choosing K:

- BIC: $\text{BIC} = -2 \log p(X|\hat{\theta}) + p \log N$
- AIC: $\text{AIC} = -2 \log p(X|\hat{\theta}) + 2p$
- Cross-validation

6.3 Latent Dirichlet Allocation (LDA)

When to use: Use LDA for topic modeling in text data when you want to discover latent topics and their distributions across documents.

Key Idea: Documents are mixtures of topics, and topics are distributions over words.

Generative Process:

1. For each document:
 - Draw topic mixture $\theta_d \sim \text{Dir}(\alpha)$
2. For each topic:
 - Draw word distribution $\phi_k \sim \text{Dir}(\beta)$
3. For each word position in each document:
 - Draw topic $z_{d,n} \sim \text{Cat}(\theta_d)$
 - Draw word $w_{d,n} \sim \text{Cat}(\phi_{z_{d,n}})$

LDA as an Admixture Model:

- Documents can belong to multiple topics
- Words within a document can come from different topics
- Extends mixture models with hierarchical structure

Inference Methods:

- Variational inference
- Gibbs sampling
- Both approximate posterior distributions for latent variables

LDA Applications:

- Document clustering
- Information retrieval
- Recommendation systems
- Feature extraction for text classification

9. Hidden Markov Models Concepts

HMM Overview:

- Models sequential data with hidden states
- States follow Markov property (depends only on previous state)
- Each state generates an observation (emission)
- Useful for temporal or sequential data

Key Components:

- Hidden states s_t : Unobserved state at time t
- Emissions x_t : Observed values at time t
- Initial state distribution: $p(s_1)$
- Transition probabilities: $p(s_t|s_{t-1})$
- Emission probabilities: $p(x_t|s_t)$

Common Problems:

- Inference: Determine $p(\text{states}|\text{observations})$
- Learning: Estimate parameters from observations
- Decoding: Find most likely state sequence

Common Applications:

- Speech recognition
- Natural language processing
- Biological sequence analysis
- Financial time series

9.1 Forward-Backward Algorithm for HMMs

When to use: Use the Forward-Backward algorithm for efficient inference in HMMs, particularly for computing marginal probabilities of hidden states.

Forward Pass (α values):

$$\alpha_t(s_t) = p(x_1, \dots, x_t, s_t) \quad (28)$$

$$\alpha_1(s_1) = p(x_1|s_1)p(s_1) \quad (29)$$

$$\alpha_t(s_t) = p(x_t|s_t) \sum_{s_{t-1}} p(s_t|s_{t-1})\alpha_{t-1}(s_{t-1}) \quad (30)$$

Backward Pass (β values):

$$\beta_t(s_t) = p(x_{t+1}, \dots, x_n|s_t) \quad (31)$$

$$\beta_n(s_n) = 1 \quad (32)$$

$$\beta_t(s_t) = \sum_{s_{t+1}} p(s_{t+1}|s_t)p(x_{t+1}|s_{t+1})\beta_{t+1}(s_{t+1}) \quad (33)$$

Inference with α and β :

$$p(x_1, \dots, x_n) = \sum_{s_t} \alpha_t(s_t)\beta_t(s_t) \quad (34)$$

$$p(s_t|x_1, \dots, x_n) \propto \alpha_t(s_t)\beta_t(s_t) \quad (35)$$

$$p(s_t, s_{t+1}|x_1, \dots, x_n) \propto \alpha_t(s_t)p(s_{t+1}|s_t) \quad (36)$$

$$\cdot p(x_{t+1}|s_{t+1})\beta_{t+1}(s_{t+1}) \quad (37)$$

Viterbi Algorithm (Best Path):

1. Initialize:

$$\delta_1(i) = p(s_1 = i)p(x_1|s_1 = i) \quad (38)$$

$$\psi_1(i) = 0 \quad (39)$$

2. Recursion:

$$\delta_t(j) = \max_i [\delta_{t-1}(i)p(s_t = j|s_{t-1} = i)] \quad (40)$$

$$\cdot p(x_t|s_t = j) \quad (41)$$

$$\psi_t(j) = \arg \max_i [\delta_{t-1}(i)p(s_t = j|s_{t-1} = i)] \quad (42)$$

3. Termination:

$$p^* = \max_i [\delta_T(i)] \quad (43)$$

$$s_T^* = \arg \max_i [\delta_T(i)] \quad (44)$$

4. Backtracking: $s_t^* = \psi_{t+1}(s_{t+1}^*)$ for $t = T - 1, \dots, 1$

HMM Training with EM:

- E-step: Compute expected state occupancy and transitions using forward-backward
- M-step: Update parameters using these expectations

10. Markov Decision Processes Concepts

MDP Overview:

- Framework for decision-making under uncertainty
- Agent takes actions in states to maximize rewards
- Environment transitions between states based on actions
- Goal: Find optimal policy (mapping from states to actions)

Key Components:

- States S : Possible situations for the agent
- Actions A : Possible choices in each state
- Transition function $p(s'|s, a)$: Probability of next state
- Reward function $r(s, a)$: Immediate reward for action
- Discount factor γ : Value of future rewards
- Policy $\pi(a|s)$: Strategy for choosing actions

Value Functions:

- State value $V^\pi(s)$: Expected return starting from state s , following policy π
- Action value $Q^\pi(s, a)$: Expected return starting from state s , taking action a , then following policy π

Planning Horizons:

- Finite horizon: Fixed number of time steps
- Infinite horizon: Unbounded time with discounting

10.1 MDP Planning Algorithms

When to use: Use these planning algorithms when you have a fully defined MDP (states, actions, transitions, rewards) and need to find the optimal policy.

Value Iteration:

1. Initialize $V(s) = 0$ for all states
2. Repeat until convergence:

$$V'(s) \leftarrow \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V(s') \right] \quad (45)$$

$$V(s) \leftarrow V'(s) \quad (46)$$

3. Extract policy: $\pi^*(s) = \arg \max_a [r(s, a) + \gamma \sum_{s'} p(s'|s, a) V^*(s')]$

Policy Iteration:

1. Initialize policy π
2. Repeat until convergence:
 - Policy Evaluation: Calculate $V^\pi(s)$

$$V^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s'} p(s'|s, \pi(s)) V^\pi(s') \quad (47)$$

- Policy Improvement:

$$\pi'(s) \leftarrow \arg \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V^\pi(s') \right] \quad (48)$$

$$\pi(s) \leftarrow \pi'(s) \quad (49)$$

Value Iteration vs. Policy Iteration:

- Value Iteration: More iterations, less computation per iteration
- Policy Iteration: Fewer iterations, more computation per iteration
- Value Iteration: $O(|S|^2|A|)$ per iteration
- Policy Evaluation: $O(|S|^3)$ for exact solution

Bellman Equations:

- Bellman Expectation Equation: $V^\pi(s) = \sum_a \pi(a|s) [r(s, a) + \gamma \sum_{s'} p(s'|s, a) V^\pi(s')]$
- Bellman Optimality Equation: $V^*(s) = \max_a [r(s, a) + \gamma \sum_{s'} p(s'|s, a) V^*(s')]$

Time Complexity Analysis

How to Calculate Time Complexity:

1. Identify basic operations

- Arithmetic operations: $O(1)$
- Array/list access: $O(1)$
- Loop iterations: $O(n)$ for n iterations
- Nested loops: $O(n \cdot m)$ for outer loop n times, inner loop m times

2. Count operations in terms of input size

- N = number of data points
- D = dimensionality of data
- K = number of clusters/classes
- T = number of iterations

3. Focus on dominant terms

- Ignore constants and lower-order terms
- Consider worst-case scenario

Example Analysis: K-means

1. For each iteration (T iterations):
 - For each data point (N points):
 - Calculate distance to each centroid (K centroids):
 - * Each distance calculation: $O(D)$ operations
 - Update K centroids: $O(N \cdot D)$ operations
2. Total complexity: $O(T \cdot N \cdot K \cdot D)$

Common Algorithm Complexities:

- Linear Regression: $O(Nd^2 + d^3)$ for optimal solution
- Gradient Descent: $O(Ndt)$ for t iterations
- K-means: $O(NKdt)$ for t iterations
- Decision Trees: $O(dN \log N)$ for training
- SVMs: $O(N^2 \text{ to } N^3)$ depending on implementation
- Neural Networks: $O(N_t \cdot \sum_\ell n_\ell n_{\ell-1})$ for training

- PCA: $O(Nd^2 + d^3)$ for computation
- HMM Forward-Backward: $O(TK^2)$ for sequence length T , K states